

# Modelo Lógico Relacional

## A4. Modelo lógico relacional

**Convenções.** Sublinhado = chave primária (nas relações de chave composta, todos os atributos sublinhados formam a PK). Sublinhado pontilhado = chave estrangeira, com o destino indicado na linha seguinte. \* = atributo opcional (aceita NULL); todos os demais são obrigatórios (NOT NULL). **UQ** = restrição de unicidade. Este esquema é derivado do MER conceitual do artefato A2 e nenhum atributo aparece aqui sem constar lá.

### 4.1 Esquema relacional

cliente(id\_cliente, nome, cpf\_cnpj, telefone\*, email\*, endereco\*, data\_cadastro)  
uq cpf\_cnpj · PK substituta · *telefone* é monovalorado (um contato preferencial por cliente).

mecanico(id\_mecanico, nome, cpf, data\_admissao, id\_supervisor\*)  
id\_supervisor → mecanico(id\_mecanico) ON DELETE SET NULL / ON UPDATE CASCADE  
uq cpf · autorrelacionamento (hierarquia de supervisão).

especialidade(id\_especialidade, nome\_especialidade)  
uq nome\_especialidade.

mecanico\_especialidade(id\_mecanico, id\_especialidade, nivel\_certificacao, data\_certificacao)  
id\_mecanico → mecanico(id\_mecanico) ON DELETE CASCADE / ON UPDATE CASCADE  
id\_especialidade → especialidade(id\_especialidade) ON DELETE RESTRICT / ON UPDATE CASCADE  
Tabela associativa do N:N **Mecânico × Especialidade**, com atributos próprios da associação.

fornecedor(id\_fornecedor, nome, cnpj, telefone\*)  
uq cnpj.

peca(id\_peca, nome, codigo\_fabricante, preco\_unitario, estoque\_atual, id\_fornecedor)  
id\_fornecedor → fornecedor(id\_fornecedor) ON DELETE RESTRICT / ON UPDATE CASCADE  
uq codigo\_fabricante — código único no sistema todo, conforme o dicionário de dados e o script 01\_ddl.sql.

veiculo(id\_veiculo, placa, marca, modelo, ano\_fabricacao, cor\*, tipo\_veiculo, id\_cliente)  
id\_cliente → cliente(id\_cliente) ON DELETE RESTRICT / ON UPDATE CASCADE  
uq placa · cx tipo\_veiculo ∈ {CARRO, MOTO} — discriminador da especialização.

carro(id\_veiculo, num\_portas, tipo\_combustivel)  
id\_veiculo → veiculo(id\_veiculo) ON DELETE CASCADE / ON UPDATE CASCADE

moto(id\_veiculo, cilindrada)  
id\_veiculo → veiculo(id\_veiculo) ON DELETE CASCADE / ON UPDATE CASCADE

servico\_catalogo(id\_servico, descricao, valor\_padrao, tempo\_estimado\_horas)

ordem\_servico(id\_os, id\_veiculo, id\_mecanico\_responsavel, data\_abertura, data\_conclusao\*, status, km\_veiculo, valor\_total)  
id\_veiculo → veiculo(id\_veiculo) ON DELETE RESTRICT / ON UPDATE CASCADE  
id\_mecanico\_responsavel → mecanico(id\_mecanico) ON DELETE RESTRICT / ON UPDATE CASCADE  
cx data\_conclusao preenchida se e somente se status ∈ {Concluída, Cancelada} (RN07).  
*valor\_total* é atributo **derivado e materializado** — ver justificativa em 4.3.

item\_ordem\_servico(id\_os, numero\_item, id\_servico, quantidade\_horas, valor\_cobrado)  
id\_os → ordem\_servico(id\_os) ON DELETE CASCADE / ON UPDATE CASCADE  
id\_servico → servico\_catalogo(id\_servico) ON DELETE RESTRICT / ON UPDATE CASCADE  
Entidade fraca: *numero\_item* é chave parcial e só identifica dentro da OS.

peca\_utilizada(id\_os, numero\_item, id\_peca, quantidade, preco\_aplicado)  
(id\_os, numero\_item) → item\_ordem\_servico(id\_os, numero\_item) ON DELETE CASCADE / ON UPDATE CASCADE  
id\_peca → peca(id\_peca) ON DELETE RESTRICT / ON UPDATE CASCADE  
Tabela associativa do N:N **ItemOrdemServiço × Peça**, com atributos próprios.

garantia(id\_garantia, id\_os, data\_inicio, data\_fim, status\_garantia, descricao\_cobertura\*)  
id\_os → ordem\_servico(id\_os) ON DELETE CASCADE / ON UPDATE CASCADE  
uq id\_os — é esta restrição que implementa a cardinalidade (0,1) do conceitual: a OS pode não ter garantia, mas nunca mais de uma. cx data\_fim > data\_inicio.

historico\_status\_os(id\_os, data\_hora, status\_anterior\*, status\_novo)  
id\_os → ordem\_servico(id\_os) ON DELETE CASCADE / ON UPDATE CASCADE  
Entidade fraca de atributo temporal: o evento não existe fora da OS; *status\_anterior* é NULL no primeiro registro (abertura da OS).

### 4.2 Decisões de mapeamento e justificativas

- **Especialização Veículo → Carro/Moto: tabela por subclasse.** Mantivemos *veiculo* com os atributos comuns e criamos *carro* e *moto* apenas com os específicos (num\_portas e tipo\_combustivel; cilindrada). A alternativa de tabela única deixaria metade das colunas sempre nula, e a alternativa de três tabelas separadas (sem a superclasse) obrigaria a UNION em toda consulta que lista veículos de um cliente — operação frequente no domínio. A especialização é **total e disjunta (t,d)**: a **disjunção** é garantida pelo esquema, porque *id\_veiculo* é ao mesmo tempo PK e FK em cada subclasse, o que impede o mesmo veículo de ter linha em *carro* e em *moto* com papéis distintos; a **totalidade** não é expressável em SQL padrão e será reforçada na aplicação e por gatilho na Etapa 2. A coluna *tipo\_veiculo* funciona como discriminador, evitando dois LEFT JOIN só para descobrir o tipo.
- **Relacionamento 1:1 Veículo-Carro/Moto: mantido separado, com PK compartilhada.** Não foi fundido porque fundir equivaleria à estratégia de tabela única, já descartada acima. O uso da mesma coluna como PK e FK garante automaticamente a cardinalidade (1,1) do lado da subclasse sem restrição adicional.

- **Autorrelacionamento em Mecânico.** Mapeado como FK *id\_supervisor* na própria tabela — o caminho correto para um 1:N unário. NULL representa o mecânico sem supervisor (líder de equipe), compatível com a cardinalidade (0,1) do conceitual. ON DELETE SET NULL evita perder o histórico da equipe quando um supervisor é desligado.
- **N:N com atributo próprio.** *Mecânico* × *Especialidade* e *ItemOrdemServiço* × *Peca* viraram tabelas associativas com PK composta pelas FKs participantes, carregando os atributos da associação (nivel\_certificacao/data\_certificacao e quantidade/preco\_aplicado). Em *peca\_utilizada* a PK tem três colunas, porque a FK para a entidade fraca já é composta: (id\_os, numero\_item) + id\_peca.
- **Entidade fraca ItemOrdemServiço.** Chave composta (id\_os, numero\_item): *numero\_item* é chave parcial e só faz sentido no contexto de uma OS — é a identificação por dependência exigida pelo projeto. A alternativa de criar um id substituto foi descartada porque apagaria justamente a dependência que o modelo quer demonstrar, e porque *peca\_utilizada* precisa referenciar o item dentro da OS.
- **Entidade fraca HistoricoStatusOS.** Chave composta (id\_os, data\_hora), pelo mesmo raciocínio: o evento de mudança de status não existe sem a OS. A precisão de data\_hora deve ser de segundos ou mais fina, para que dois eventos da mesma OS não colidam na PK.
- **Chave natural vs. chave substituta, relação a relação.** Ver o quadro de 4.4. O critério geral foi: substituta em toda entidade forte, porque os candidatos naturais (cpf, cnpj, placa) são corrigíveis — uma placa recadastrada obrigaria atualização em cascata em *ordem\_servico* — e porque inteiros dão índices menores; mantivemos o candidato natural como UNIQUE, de modo que a integridade do domínio não se perde. Nas entidades fracas e associativas a chave é composta pelas dependências, que já são naturais àquele contexto.
- **Cardinalidade (0,1) de Garantia.** Implementada por *id\_garantia* substituta + UNIQUE(id\_os). A alternativa seria usar *id\_os* diretamente como PK de *garantia*; optamos pela substituta porque a garantia é referenciada por número próprio no atendimento ao cliente. Sem o UNIQUE, o esquema aceitaria duas garantias para a mesma OS e deixaria de refletir o conceitual — por isso a restrição é obrigatória, não opcional.

### 4.3 Desnormalização deliberada: *ordem\_servico.valor\_total*

*valor\_total* é derivável — é a soma de *item\_ordem\_servico.valor\_cobrado* da OS, isto é, o total de **mão de obra** dos itens de serviço executados; peças consumidas são apuradas à parte, em *peca\_utilizada*, e não compõem esta coluna. Essa é a mesma regra usada no script `02_carga.sql` e verificável em qualquer OS carregada. O atributo está assinalado como derivado no MER (elipse tracejada). Optamos por materializá-lo porque as telas de listagem de OS e os relatórios de faturamento do período exibem o total de dezenas a centenas de ordens de uma vez; calculá-lo a cada leitura exigiria um agregado com junção por linha listada. A consistência será mantida por gatilho de INSERT/UPDATE/DELETE sobre *item\_ordem\_servico* na Etapa 2, e não pela aplicação — assim nenhum caminho de escrita pode deixar o valor defasado. Essa é a única redundância deliberada do esquema; todas as demais relações estão livres de dependências parciais e transitivas (demonstração no artefato A5).

### 4.4 Quadro de chaves

| Relação                | Chave primária                  | Tipo                         | Candidata natural mantida como UNIQUE |
|------------------------|---------------------------------|------------------------------|---------------------------------------|
| cliente                | id_cliente                      | substituta                   | cpf_cnpj                              |
| mecanico               | id_mecanico                     | substituta                   | cpf                                   |
| especialidade          | id_especialidade                | substituta                   | nome_especialidade                    |
| mecanico_especialidade | (id_mecanico, id_especialidade) | composta por FKs             | —                                     |
| fornecedor             | id_fornecedor                   | substituta                   | cnpj                                  |
| peca                   | id_peca                         | substituta                   | codigo_fabricante                     |
| veiculo                | id_veiculo                      | substituta                   | placa                                 |
| carro / moto           | id_veiculo                      | herdada da superclasse       | —                                     |
| servico_catalogo       | id_servico                      | substituta                   | descricao                             |
| ordem_servico          | id_os                           | substituta                   | —                                     |
| item_ordem_servico     | (id_os, numero_item)            | dependência de identificação | —                                     |
| peca_utilizada         | (id_os, numero_item, id_peca)   | composta por FKs             | —                                     |
| garantia               | id_garantia                     | substituta                   | id_os                                 |
| historico_status_os    | (id_os, data_hora)              | dependência de identificação | —                                     |

### 4.5 Rastreabilidade MER → esquema relacional

| Construção no MER (A2)                                                      | Resultado no modelo lógico                     |
|-----------------------------------------------------------------------------|------------------------------------------------|
| 9 entidades fortes                                                          | 9 relações correspondentes                     |
| 2 entidades fracas (ItemOrdemServiço, HistóricoStatusOS)                    | 2 relações com PK composta pela dependência    |
| Especialização (t,d) Veículo → Carro/Moto                                   | veiculo + carro + moto (tabela por subclasse)  |
| 2 relacionamentos N:N com atributo                                          | mecanico_especialidade, peca_utilizada         |
| Relacionamentos 1:N (possui, fornece, gera, é executada por, é referente a) | FK no lado (1,1)                               |
| Relacionamento unário supervisiona (0,N)/(0,1)                              | FK id_supervisor em mecanico                   |
| Relacionamento (0,1) recebe                                                 | FK id_os em garantia + UNIQUE(id_os)           |
| Atributo derivado valor_total                                               | coluna materializada, com justificativa em 4.3 |